

## Informatics II, Spring 2026, Exercise 11

Publication of exercise: May 18, 2026

Publication of solution: May 26, 2026

Exercise classes: May 26 - May 29, 2026

### Task 1

Consider the following undirected Graph  $G_1$ :

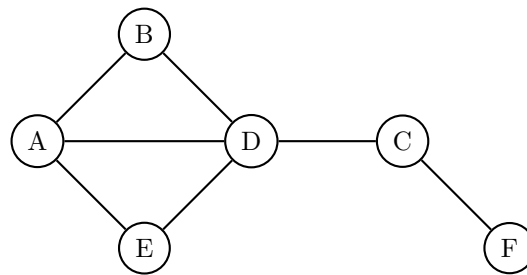


Figure 1:  $G_1$

1. Find a set of vertices in  $G_1$  build up a cycle:
2. Find a sequence in  $G_1$  could be the result of a BFS starting from vertex C:
3. Find a sequence in  $G_1$  could be the result of a DFS starting from vertex C:

## Graphs

Consider a Graph  $G_2$  given by the following adjacency matrix.  
(To answer the questions, it may help you to draw out the graph  $G_2$ .)

$$G_2 \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Mark as True or False and justify your answer:

4. The Graph  $G_2$  is acyclic.
5. The Graph  $G_2$  is a directed Graph.

## Task 2: Finding cycle in undirected Graph

Write a pseudocode algorithm that given a Graph  $G$  returns TRUE if the Graph contains a cycle and FALSE if it does not.

Every vertex  $v \in V$  has the following properties:

- $v.adj$ : A list with all vertices adjacent to  $v$
- $v.col$ : The coloring of the vertex (White, Grey, Black)
- $v.pred$ : The predecessor of  $v$

You are free to use further properties as it suits your algorithm.

Hint: Try to solve this problem with a DFS approach.

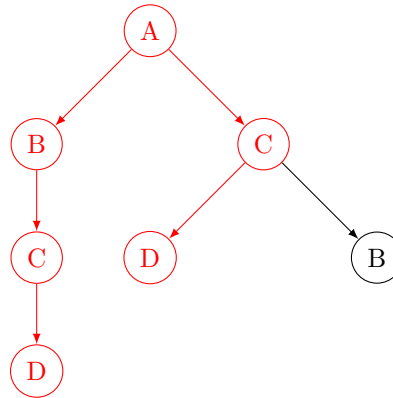
## Task 3: Count all possible Paths between two Vertices

Write a pseudocode algorithm that given the adjacency Matrix  $M$  of a Graph, a source vertex  $s$  and a destination vertex  $t$  returns the number of all possible simple paths between those vertices. Be aware that the Graph may contain cycles and you are not allowed to visit a vertex twice.

Hint: Use a DFS approach to enumerate all possible paths from  $s$  to  $t$ .

For example, for the Graph given below there are 2 paths from Vertex A to D marked in red. Every branch in the Tree represents a possible simple path from vertex A. Keep track of the visited vertices in the current branch by using a list or similar to avoid visiting a vertex twice which would lead to a cycle.

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$



## Task 4: k-hop

Consider a directed graph  $G = (V, E)$  with nodes  $v \in V$  and  $u \in V$ . Node  $u$  is a  **$k$ -hop neighbor** of node  $v$  if there is at least one path from  $v$  to  $u$  and the minimum distance from  $v$  to  $u$  is  $k$ . The distance from  $v$  to  $u$  is infinite if there is no path from  $v$  to  $u$ .

Let  $|V|$  be the number of nodes in  $G$ . Each node is identified by a unique integer between 1 and  $|V|$ . A directed edge  $(v, u) \in E$  is represented as a pair of nodes and denotes an edge from node  $v \in V$  to node  $u \in V$ .

1. Assume a graph  $G = (V, E)$  with vertices  $V = \{1, 2, 3, 4, 5, 6\}$  and edges  $E = \{(1, 2), (1, 4), (1, 5), (2, 3), (2, 5), (3, 1), (4, 6)\}$ . Draw the adjacency matrix of  $G$ .
2. Let  $k = 2$ . Determine all 2-hop neighbors of node 1? Explain your approach.
3. Assume a directed graph  $G$  with  $n$  nodes. An adjacency matrix  $a[n, n]$  is used to represent the edges of the graph. Give a pseudocode algorithm that prints all  $k$ -hop neighbors of node  $v$ .